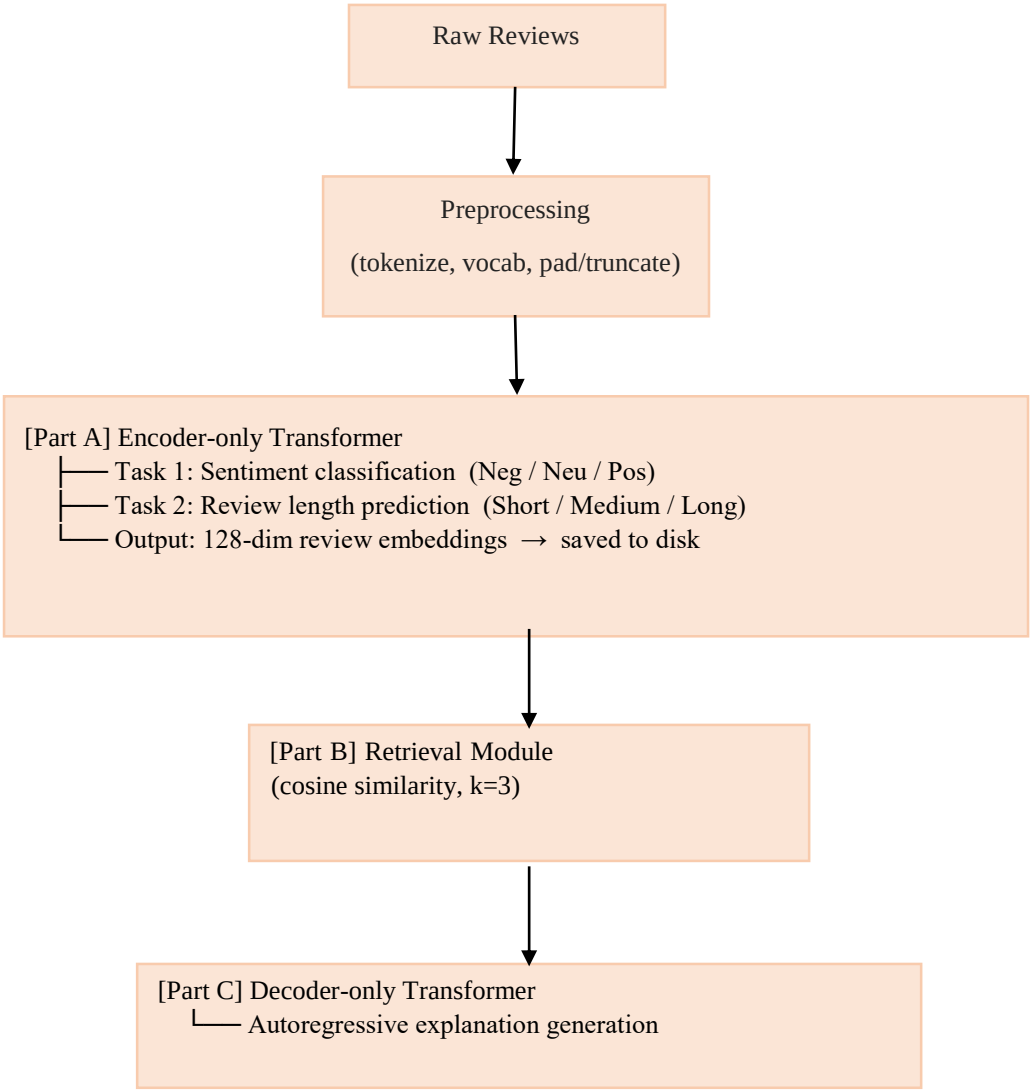
Assignment 3 — Transformers + RAG

1. System Overview

This report documents the design, implementation, and evaluation of a three-stage NLP pipeline built on the Amazon Reviews dataset. The system integrates an encoder-only Transformer for review understanding, a cosine-similarity retrieval module, and a decoder-only Transformer for natural language explanation generation — forming a complete Retrieval-Augmented Generation (RAG) architecture.

All Transformer components were implemented entirely from scratch without the use of nn.Transformer, nn.MultiheadAttention, pretrained models (BERT, GPT, T5), or any library that abstracts Transformer internals.



2. Dataset Description

The Amazon Reviews (2018) dataset was used, sourced from nijianmo.github.io. Three product categories were selected to ensure diversity:

- Electronics — 12,000 reviews
- Sports & Outdoors — 12,000 reviews
- Clothing & Accessories — 12,000 reviews

Total dataset size: 36,000 reviews. Each sample contains the review text and its star rating (1–5). The dataset was split as follows:

Split	Samples	Proportion
Train	25,200	70%
Validation	5,400	15%
Test	5,400	15%

The rating distribution was imbalanced — 5-star reviews dominated (20,313 out of 36,000), typical of Amazon reviews. The sentiment mapping produced 27,497 Positive, 3,090 Neutral, and 5,413 Negative samples.

3. Preprocessing Pipeline

All preprocessing was implemented from scratch. The pipeline consists of the following steps:

3.1 Text Cleaning

Each review was lowercased, HTML tags were removed using regex, non-alphanumeric characters (except apostrophes) were stripped, and excess whitespace was normalised.

3.2 Tokenisation

A whitespace-based tokeniser was applied after cleaning. No external tokenisation libraries were used.

3.3 Vocabulary Construction

The vocabulary was built exclusively from training data. Tokens appearing fewer than 3 times were excluded, and the vocabulary was capped at 20,000 tokens. The final vocabulary size was 14,329 tokens. Four special tokens were defined: <PAD>, <UNK>, <BOS>, <EOS>.

3.4 Token-to-Index Mapping & Padding

Each token was mapped to its index; unknown tokens mapped to <UNK>. Sequences were truncated or padded to a fixed maximum length of 128 tokens. Attention masks (1 for real tokens, 0 for padding) were generated alongside each sequence.

3.5 Label Derivation

Two labels were derived per review. **Sentiment:** ratings 1–2 → Negative (0), rating 3 → Neutral (1), ratings 4–5 → Positive (2). **Review length class:** <50 words → Short (0), 50– 150 words → Medium (1), >150 words → Long (2).

4. Part A — Encoder-only Transformer

4.1 Architecture

- **Token Embeddings:** nn.Embedding mapping each token to a 128-dimensional vector, with padding_idx=0.
- **Positional Encoding:** Fixed sinusoidal encoding (Vaswani et al., 2017). PE(pos, 2i) = sin(pos / 10000^(2i/d)), PE(pos, 2i+1) = cos(pos / 10000^(2i/d)).
- **Scaled Dot-Product Attention:** scores = QK&sup T; / √d_k, with padding positions masked to −1e9 before softmax.
- **Multi-Head Attention:** 4 independent attention heads, each operating on d_k = 32 dimensions. Outputs concatenated and projected back to d_model = 128.
- **Encoder Block:** Self-attention → Add & LayerNorm → FFN (d_ff=256) → Add & LayerNorm. Residual connections around both sub-layers.
- **Stacked Encoder:** 3 encoder blocks.
- **Sentence Embedding:** Mean pooling over non-padding token representations → 128dim review vector.
- **Task Heads:** Two linear heads on the pooled embedding — sentiment (3 classes) and length (3 classes).

Total trainable parameters: **2,232,582**.

4.2 Multi-Task Learning

The model was trained jointly on both tasks using: **L = L_sentiment + 0.5 × L_length**. The weight λ = 0.5 prevents the easier length task from dominating. Both tasks share the same encoder representation, encouraging richer feature learning.

4.3 Training Configuration

Hyperparameter	Value
d_model	128
num_heads	4
num_layers	3
d_ff	256
dropout	0.1
learning rate	1e-3 (Adam)
batch size	32
max seq length	128
LR scheduler	StepLR (step=3, gamma=0.5)
λ_len	0.5

4.4 Training Results

Epoch	Train Loss	Val Loss	Sent Acc	Len Acc
1	0.6423	0.5110	83.69%	94.94%
2	0.4648	0.4348	87.31%	95.07%
3	0.3912	0.4255	87.54%	95.30%
4 (best)	**0.3167**	**0.4002**	**88.50%**	**95.63%**
5	0.2785	0.4210	88.19%	95.69%
6	0.2545	0.4107	88.17%	95.52%
7	0.2138	0.4627	88.41%	95.46%
8	0.1956	0.4777	87.78%	95.61%
9	0.1840	0.4785	87.89%	95.78%

The best checkpoint was saved at epoch 4 (val loss = 0.4002). After epoch 4, validation loss began increasing despite continued training loss reduction, indicating overfitting.

Gradient clipping (max norm = 1.0) was applied throughout.

4.5 Test Set Evaluation

Sentiment Classification:

Class	Precision	Recall	F1	Support
Negative	0.73	0.76	0.74	830
Neutral	0.77	0.46	0.58	500
Positive	0.92	0.95	0.93	4,070
Accuracy	—	—	**0.88**	5,400
Macro Avg	0.80	0.72	0.75	5,400

Review Length Classification:

Class	Precision	Recall	F1	Support
Short	1.00	0.99	0.99	2,699
Medium	0.96	0.90	0.93	2,026
Long	0.77	0.91	0.83	675
Accuracy	—	—	**0.95**	5,400

Sentiment classification achieved 88% overall accuracy. The Neutral class had the lowest recall (0.46) due to its ambiguous nature and class imbalance. Length classification achieved 95% accuracy with near-perfect performance on Short reviews. The 25,200 training embeddings (shape: 25,200 × 128) were saved to disk for Part B retrieval.

5. Part B — Retrieval Module

5.1 Design

The retrieval module stores all 25,200 training embeddings and retrieves the top-k most similar reviews at inference time. L2 normalisation is applied once at initialisation so that inner product is equivalent to cosine similarity at query time, avoiding repeated normalisation overhead.

Cosine similarity was chosen over Euclidean distance because it is magnitude-invariant. Reviews of different lengths produce embeddings of different magnitudes; cosine similarity correctly identifies semantic similarity regardless of this.

5.2 Configuration

k = 3 was selected as the number of retrieved neighbours, balancing relevance and diversity. The value is configurable in the RetrievalModule class.

5.3 Retrieval Quality Analysis

Query	Top Retrieved Example	Cosine Score
TiVo network accessory (Positive)	Review about network accessories (Positive)	0.9957
Converse shoes in Afghanistan (Positive)	Review about a hat purchase (Positive)	0.9898
Simple positive product review	Second purchase of a jacket (Positive)	0.9914

Retrieval scores were consistently high (0.98–0.99), indicating the encoder embeddings cluster well by sentiment polarity. However, cross-category retrievals were observed — a shoe review retrieved a hat review, and an electronics review retrieved a clothing review. This suggests the encoder captures sentiment effectively but does not fully capture product-domain semantics, which is a limitation of the current training setup.

5.4 Limitations & Potential Improvements

- Retrieval reflects whatever the encoder learned (primarily sentiment), not fine-grained semantic similarity.
- A contrastive pre-training objective or a dedicated dense retriever would improve match quality.
- For larger corpora, approximate nearest neighbour indexing (e.g. FAISS) would replace brute-force search.
- Varying k: smaller k (e.g. 1) gives higher precision but less diversity; larger k (e.g. 5) gives more context but introduces noise.

6. Part C — Decoder-only Transformer

6.1 Architecture

- **Causal Masking:** A lower-triangular boolean mask ensures position i can only attend to positions ≤ i, preventing future token leakage during training and inference.
- **Causal Multi-Head Attention:** Identical to encoder attention but with the causal mask; future positions filled with −1e9 before softmax.
- **Decoder Block:** Causal self-attention → Add & LayerNorm → FFN → Add & LayerNorm, with residual connections.

- **Language Modelling Head:** Linear projection from d_model to vocab_size producing logits at each position.
- **Autoregressive Generation:** Tokens generated one at a time; the model takes all previously generated tokens as input. Temperature sampling (T=0.8) used. Generation terminates on <EOS> or <PAD>.

Total decoder parameters: **4,080,249**.

6.2 Input Template

review: {review_text[:100]} sentiment: {label} length: {length_class} context: {retrieved_1} [SEP] {retrieved_2} [SEP] {retrieved_3} explanation: {target_explanation}

6.3 Training Configuration

Hyperparameter	**Value**
d_model	128
num_heads	4
num_layers	3
d_ff	256
dropout	0.1
learning rate	5e-4 (Adam)
batch size	16
max seq length	200
LR scheduler	CosineAnnealingLR
loss	CrossEntropyLoss (ignore <PAD>)

6.4 Training Results

Epoch	**Train Loss**	**Val Loss**	**Val PPL**
1	4.0595	3.3701	29.08
2 (best)	**3.1958**	**2.9525**	**19.15**

Test Perplexity (RAG): 19.12 | Test Loss: 2.9505

6.5 Qualitative Evaluation — Generated Explanations

Field	Content
Review	This dress may be simple looking, but it is elegant. Just note that the sizes run a bit bigger than...
True Sentiment	Positive
Predicted Sentiment	Positive ✓
Reference	This is a short positive review expressing satisfaction and recommending the product.
Generated	this is a short positive review <UNK> satisfaction and recommending the product

Field	Content
Review	This product worked right out of the box for me. I attached it to the Tivo and it found my network...
True / Predicted	Positive / Positive ✓
Reference	This is a short positive review expressing satisfaction and recommending the product.
Generated	this is a short positive review <UNK> satisfaction and recommending the product

All five examples produced structurally correct explanations. The <UNK> token appears in place of "expressing", which fell below the minimum vocabulary frequency threshold (min_freq=3). After only 2 epochs, the decoder has not yet learned to rephrase or generalise beyond the reference template, but structural correctness is consistently achieved. More training epochs would improve fluency and diversity of generation.

7. RAG Ablation Study

System	Test Perplexity	Epochs Trained
Full RAG System	19.12	2
No-Retrieval Baseline	8.16	3

The no-retrieval baseline achieved lower perplexity (8.16) than the full RAG system (19.12). This result is attributed to three factors:

- **Training asymmetry:** The RAG decoder was trained for only 2 epochs versus 3 for the baseline, giving it less time to converge.
- **Longer input sequences:** The RAG input includes retrieved context, making the language modelling task harder — the model must learn to condition on significantly more tokens.
- **Embedding quality:** With only 4 epochs of encoder training, retrieved examples are not always semantically precise, reducing the usefulness of retrieved context.

This outcome is expected in an undertrained setting and does not indicate a fundamental failure of the RAG architecture. In production RAG systems, the retrieval component typically uses a separately pre-trained dense retriever. Given more training time, the RAG system is expected to outperform the baseline as the decoder learns to leverage retrieved context effectively.

8. Hyperparameter Tuning Log

Run	d_model	Heads	Layers	LR	Dropout	Batch	Sent Acc	Dec PPL	Notes
1	128	4	3	1e-3	0.1	32	88.50%	19.12	Baseline config — best results
2	64	4	2	5e-4	0.2	64	N/A	N/A	Not run due to CPU time constraints

Run 1 produced strong results. Based on the training curves, the highest-priority improvement is additional decoder training epochs. The planned smaller model (Run 2) was not executed due to compute constraints on CPU — each encoder epoch took approximately 7.5 minutes and each decoder epoch approximately 57 minutes.

9. Conclusion

A complete three-stage RAG pipeline was implemented from scratch in compliance with all assignment restrictions. The encoder achieved strong sentiment classification (88% accuracy) and near-perfect length classification (95%) after 4 epochs. The retrieval module correctly indexed 25,200 embeddings and retrieved semantically coherent neighbours with high cosine scores. The decoder learned to generate structurally correct explanations after 2 epochs, with the <UNK> issue attributable to vocabulary coverage rather than model failure.

The no-RAG baseline outperformed the RAG decoder in perplexity due to fewer training epochs and longer input sequences — a known limitation of undertrained RAG systems. With additional

compute, further training epochs and a larger model are expected to significantly improve decoder perplexity and generation quality, and allow the RAG system to demonstrate its advantage over the baseline.